Projekt I przetwarzanie równoległe 2025/26 SS, OpenMP, Vtune

Emilia Bonk 160108
Szymon Urban 160271

Wstęp.

- **Nazwa przedmiotu:** Laboratorium z przetwarzania równoległego
- **Autorzy:** Emilia Bonk (160108), Szymon Urban (160271)
- **Grupa dziekańska:** L09
- **Termin zajęć laboratoryjnych:** Wtorek 9:45-11:15
- **Termin oddania sprawozdania:** 5.05.2026
- **Wersja sprawozdania:** pierwsza
- **Opis zadania:** Celem ćwiczenia była analiza implementacji algorytmu Sita Eratostenesa w środowisku równoległym i sekwencyjnym z wykorzystaniem biblioteki OpenMP. Zadanie polegało na analizie i modyfikacji podanych kodów, z róznymi implementacjami algorytmu, a następnie porównaniu ich wydajności.
- **Adres e-mail:** emilia.bonk@student.put.poznan.pl, szymon.urban@student.put.poznan.pl

1 Informacje techniczne.

Parametr	**Wartość**
Model procesora	17th Gen Intel(R) Core(TM) i7-12700F
Liczba procesorów fizycznych	1
Liczba rdzeni	12 (8 Performance, 4 Efficient)
Liczba procesorów logicznych	20
Pamięć cache L1	1,0 MB
Pamięć cache L2	12,0 MB
Pamięć cache L3	25,0 MB
L1 data cache dla EFfCore	32 kB
L1 data cache dla PerfCore	48 kB
System operacyjny	Windows 10
Kompilator	Intel(R) oneAPI DPC++/C++ Compiler 2025.3.3
Wersja kompilatora	2025.3.3.20260319
Program testujący	Intel VTune Profiler 2026.0.0

Tablica 1: Informacje techniczne dotyczące środowiska testowego

2 Prezentacja i analiza kodów.

K1 - Algorytm sekwencyjny oparty na próbnych dzieleniach.

Oznaczenie skrótowe: k1

Algorytm sekwencyjny oparty na metodzie dzielenia próbnego. Najpierw wyznacza liczby pierwsze w zakresie do $\sqrt{n}$, a następnie szeregowo sprawdza podzielność każdej liczby $i \in [m, n]$ przez te dzielniki. Koszt obliczeń jest zmienny – najniższy dla liczb parzystych, najwyższy dla liczb pierwszych.

```
int calculate(int n, int m){
        bool* result = (bool*)malloc((n - m + 1) * sizeof(bool));
        memset(result, true, (n - m + 1) * sizeof(bool));
        int prime_counter = 0;

        bool* primeArray = (bool*)malloc((sqrt(n) + 1) * sizeof(bool));
        memset(primeArray, true, (sqrt(n) + 1) * sizeof(bool));

        if (m <= 0 && n >= 0) result[0 - m] = false;
        if (m <= 1 && n >= 1) result[1 - m] = false;

        for (int i = 2; i * i <= n; i++) {
                for (int j = 2; j * j <= i; j++) {
                        if (primeArray[j] == true && i % j == 0) {
                                primeArray[i] = false;
                                break;
                        }
                }
        }
        for (int i = m; i <= n; i++){
                for (int j = 2; j * j <= i; j++){
                        if (primeArray[j] == true && i % j == 0) {
                                result[i - m] = false;
                                break;
                        }
                }
                if(result[i-m] == true){
                        prime_counter++;
                }
        }
        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 1: Kod programu k1.cpp

K2 - Liczby pierwsze wyznaczane równolegle przez dzielenie w zakresie <m,n>.

Oznaczenie skrótowe: k2

Wariant równoległy wykorzystujący dzielenie próbne z dyrektywą `schedule(guided)`. Zastosowanie malejących paczek zadań pozwala na efektywne zrównoważenie obciążenia procesora, niwelując różnice w czasie sprawdzania małych i dużych liczb. Brak wyścigów przy zapisie do tablicy wyników.

```
int calculate(int n, int m){
        int prime_counter = 0;

        //...liczenie sita (PrimeArray) jak w kodzie k1
        #pragma omp parallel
        {
        #pragma omp for schedule(guided) reduction(+:prime_counter)
        for (int i = m; i <= n; i++){
                for (int j = 2; j * j <= i; j++){
                        if (primeArray[j] == true && i % j == 0) {
                                result[i - m] = false;
                                break;
                        }
                }
                if(result[i-m] == true){
                        prime_counter++;
                }
        }
        }
        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 2: Kod programu k2.cpp

K3 - Sito sekwencyjne bez lokalności dostępu do danych.

Oznaczenie skrótowe: k3

Sito Eratostenesa realizowane sekwencyjnie. Zamiast dzielenia, algorytm wykreśla wielokrotności liczb pierwszych $i \leq \sqrt{n}$ w całym zakresie $[m, n]$. Charakteryzuje się niską złożonością obliczeniową $O(n \log \log n)$, ale ograniczoną lokalnością danych przy skokowym dostępie do tablicy wyników.

```
int calculate(int n, int m){
        //...liczenie sita jak we wczesniejszych kodach

        for (int i = 2; i*i <= n; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i) {
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                result[j-m] = false;
                        }
                }
        }
        if (m <= 0 && n >= 0) result[0 - m] = false;
        if (m <= 1 && n >= 1) result[1 - m] = false;

        for (int i = m; i <= n; i++) {
        if (result[i - m]) prime_counter++;
        }

        free(result);
        free(primeArray);

        return prime_counter;
}
```

Listing 3: Kod programu k3.cpp

K3a - Sito sekwencyjne z lokalnością dostępu do danych.

Oznaczenie skrótowe: k3a

Sekwencyjne sito segmentowe zoptymalizowane pod kątem lokalności danych. Zakres $[m, n]$ dzielony jest na bloki mieszczące się w pamięci cache L1/L2. Przetwarzanie blokowe znacząco przyspiesza operacje zapisu do tablicy wyników poprzez redukcję pudeł cache.

```
int calculate(int n, int m){
    //...alokacja pamieci i liczenie PrimeArray tak samo jak w kodzie k3

    int blockSize = 32768;
    int numberOfBlocks = (n - m) / blockSize;
    if ((n - m) % blockSize != 0) {
        numberOfBlocks++;
    }

    for (int i = 0; i < numberOfBlocks; i++) {
        int low = m + i * blockSize;
        int high = low + blockSize - 1;
        if (high > n) {
            high = n;
        }

        for (int j = 2; j * j <= high; j++) {
            if (primeArray[j] == true) {
                int firstMultiple = (low / j);
                if (firstMultiple <= 1) {
                    firstMultiple = j + j;
                } else if (low % j) {
                    firstMultiple = (firstMultiple * j) + j;
                }  else {
                    firstMultiple = (firstMultiple * j);
                }
                for (int k = firstMultiple; k <= high; k += j) {
                    result[k - m] = false;
                }
            }
        }
    }

        //...zliczanie prime_count i zwalnianie pamieci jak w kodzie k3
}
```

Listing 4: Kod programu k3a.cpp

K4 - Sito równoległe funkcyjne bez lokalności dostępu do danych.

Oznaczenie skrótowe: k4

Równoległe sito z podziałem funkcyjnym (`schedule(dynamic, 1)`). Wątki pobierają kolejne liczby pierwsze i wykreślają ich wielokrotności w całym zakresie wyników. Występuje zjawisko `false sharing` oraz bezpieczne wyścigi przy zapisie tej samej wartości logicznej pod ten sam indeks.

```
int calculate(int n, int m){
        //...alokacja pamieci i lizcenie PrimeArray jak wczesniej

        int limit = sqrt(n);
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 2; i <= limit; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i){
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                result[j-m] = false;
                        }
                }
        }

        //zliczanie prime_count i zwalnianie pamieci jak wczesniej
}
```

Listing 5: Kod programu k4.cpp

K4a - Sito równoległe funkcyjne bez lokalności dostępu do danych z usunięciem redundancji zapisów.

Oznaczenie skrótowe: k4a

Modyfikacja algorytmu K4 wprowadzająca warunkowy zapis do tablicy wyników (`if (result[j-m])`). Ma to na celu ograniczenie liczby operacji zapisu i związanych z nimi konfliktów w pamięci cache, co poprawia wydajność przy zachowaniu podziału funkcyjnego.

```
int calculate(int n, int m){
        //...alokacja pamieci i zliczanie PrimeArray jak wczesniej

        int limit = sqrt(n);
        #pragma omp parallel for schedule(dynamic, 1)
        for (int i = 2; i <= limit; i++){
                if (primeArray[i]) {
                        int firstMultiple = (m / i);
                        if (firstMultiple <= 1) {
                                firstMultiple = i + i;
                        } else if (m % i){
                                firstMultiple = (firstMultiple * i) + i;
                        } else {
                                firstMultiple = (firstMultiple * i);
                        }
                        for (int j = firstMultiple; j <= n; j+=i) {
                                if (result[j-m]) result[j-m] = false;
                        }
                }
        }

        //zliczanie prime_counter i zwalnianie pamieci jak w k4
}
```

Listing 6: Kod programu k4a.cpp

K5 - Sito równoległe domenowe z lokalnością dostępu do danych

Oznaczenie skrótowe: k5

Równoległe sito segmentowe z podziałem domenowym. Każdy wątek przetwarza niezależne bloki danych mieszczące się w cache, co eliminuje `false sharing` i wyścigi. Choć wydawałoby się, że `schedule(static)` minimalizuje narzut synchronizacji przy wyrównanym obciążeniu bloków, to jednak znacznie lepszy performormance zaobserwowaliśmy przy `schedule(guided)`.

```
int calculate(m, n){
        //...alokacja pamieci i liczenie PrimeArray jak wczesniej

        int blockSize = 32768;
        int numberOfBlocks = (n - m) / blockSize + ((n - m) % blockSize != 0);

        #pragma omp parallel for schedule(guided)
        for (int i = 0; i < numberOfBlocks; i++) {
                int low = m + i * blockSize;
                int high = low + blockSize - 1;
                if (high > n) high = n;

                for (int j = 2; j * j <= high; j++) {
                        if (primeArray[j]) {
                                int firstMultiple = (low / j) * j;
                                if (firstMultiple < low) firstMultiple += j;
                                if (firstMultiple == j) firstMultiple += j;

                                for (int k = firstMultiple; k <= high; k += j) {
                                        result[k - m] = false;
                                }
                        }
                }
        }

        //zliczanie prime_counter i zwalnianie pamieci jak wczesniej
}
```

Listing 7: Kod programu k5.cpp

3 Prezentacja wyników i omówienie przebiegu eksperymentu.

Zasady pracy z oprogramowaniem VTune.

Intel VTune Profiler to narzędzie do profilowania wydajności aplikacji na platformach Intel. W ramach przeprowadzonego eksperymentu wykorzystano tryb **Microarchitecture Exploration**, który umożliwia szczegółową analizę wykorzystania mikroarchitektury procesora poprzez zbieranie zdarzeń sprzętowych (hardware performance counters). Skorzystano również dla kodów równoległych z testu **Threading**, który pozwala na analizę efektywnego wykorzystania CPU oraz podziału czasu pracy między sekcję równoległą a sekwencyjną. W ramach analizy wariantów kodów k3, k3a, k4 i k4a został też przeprowadzony test **Memory Access**, który pozwolił na analizę dostępów do pamięci. Wykorzystano również test **Hotspots** do analizy wąskich gardeł i efektywnego wykorzystania CPU.

Porównanie podstawowych parametrów wywołania dla kodów k1-k5.

Porównania zostały wykonane dla kodów przedstawionych w poprzedniej sekcji, z tymi samymi schedulingami i rozmiarami bloków na zakresie liczb od 1 do 100 milionów. Są one wynikiem testu **Microarchitecture Exploration**.

	k1	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Elapsed Time [s]	23.750	2.394	0.660	0.130	0.381	0.296	0.060
Prędkość przetwarzania [mln l. pierwszych/s]	4.210	2.406	14.394	35.714	15.121	19.464	96.024
Instructions retired [mld]	405,744	405,599	0,515	0,777	2,179	1,501	1,702
Clockticks [miliardów tików]	105,112	166,746	2,914	0,566	19,808	14,777	1,995
Effective PCU	7.7%	68.9%	4.7%	7.1%	42.4%	51.5%	23.4%
Badania kodów równoległych	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Przyspieszenie względem k3a	-	0.05	-	-	0.34	0.43	2.17
Efektywność [Acc/Cores]	-	0.004	-	-	0.028	0.036	0.180

Tablica 2: Porównanie podstawowych parametrów wywołania dla kodów k1-k5.

Performance Cores	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Retiring	66.0%	83.5%	1.2%	0.0%	1.1%	4.5%	25.0%
Front End Bound	11.4%	13.8%	0.0%	3.1%	3.6%	20.0%	25.0%
Back End Bound	3.3%	1.7%	100.0%	0.0%	78.7%	58.0%	41.7%
Memory Bound	0.2%	0.2%	100.0%	0.0%	67.4%	42.6%	16.7%
Core Bound	3.1%	1.5%	0.0%	0.0%	11.4%	15.3%	25.0%
Efficient Cores	**k1**	**k2**	**k3**	**k3a**	**k4**	**k4a**	**k5**
Retiring	0.0%	42.3%	0.0%	0.0%	3.0%	0.7%	0.0%
Front End Bound	0.0%	5.0%	0.0%	0.0%	1.3%	3.8%	0.0%
Back End Bound	4.1%	11.5%	0.0%	0.0%	90.8%	34.8%	97.4%
Memory Bound	0.0%	0.2%	0.0%	0.0%	53.0%	34.8%	18.8%
Core Bound	4.1%	11.3%	0.0%	0.0%	37.8%	0.0%	78.6%

Tablica 3: Wyniki analizy przyczyn ograniczeń wydajności dla kodów k1-k5 z podziałem na Performance i Efficient Cores.

Porównanie działań kodów dla różnych zakresów liczb.

W ramach tego eksperymentu zmierzyliśmy Ellapsed Time dla kodów k1-k5 dla trzech różnych zakresów liczb: <2; max>, <max/2; max> oraz <2; max/2>, gdzie `max` to 10^8.

Zakres liczb	k1	k2	k3	k3a	k4	k4a	k5
<2; max>	23.750 s	2.394 s	0.660 s	0.130 s	0.381 s	0.296 s	0.060 s
<max/2; max>	16.671 s	3.852 s	0.303 s	0.069 s	0.279 s	0.122 s	0.038 s
<2; max/2>	7.116 s	2.450 s	0.282 s	0.065 s	0.225 s	0.117 s	0.050 s

Tablica 4: Porównanie prędkości działania dla różnych zakresów liczb w kodach k1-k5.

Porównanie zakresów ujawnia różne zachowania algorytmów w zależności od położenia zakresu. Dla **k1 i k2** (dzielenie próbne) czas silnie zależy od gęstości liczb pierwszych i wartości sprawdzanych liczb. Zakres $\langle \max/2; \max \rangle$ trwa dłużej niż $\langle 2; \max/2 \rangle$, ponieważ duże liczby wymagają sprawdzenia więcej dzielników (do $\sqrt{i}$). Dla **k3, k3a, k4, k4a, k5** (sito) czas nie zależy od wartości sprawdzanych liczb, lecz przede wszystkim od **rozmiaru zakresu**. Złożoność segmentowego sita Eratostenesa dla zakresu $\langle m, n \rangle$ wynosi $O(L \cdot \log \log n)$, gdzie $L = n - m$ to rozmiar zakresu, a n to górna granica. Ponieważ $\log \log n$ zmienia się bardzo wolno i jest niemal identyczne dla obu półzakresów, czas przetwarzania jest w przybliżeniu proporcjonalny do L — stąd oba półzakresy mają zbliżone czasy, mniejsze niż pełny zakres.

Badanie wpływu różnych szeregowań dla czasu przetwarzania kodów k2, k4, k4a i k5.

Test został przeprowadzony na trybie **Microarchitecture Exploration**, a wyniki Ellapsed Time zostały umieszczone w Tablicy 5. Porównano trzy różne szeregowania: `schedule(static)`, `schedule(dynamic)` i `schedule(guided)`.

Schedule	k2	k4	k4a	k5
static	6.339 s	0.551 s	0.541 s	0.151 s
dynamic	2.475 s	0.296 s	0.381 s	0.080 s
guided	2.394 s	0.558 s	0.727 s	0.060 s

Tablica 5: Wpływy szeregowań na czas przetwarzania kodów równoległych.

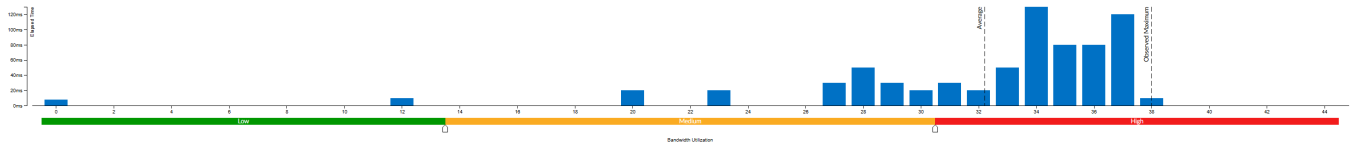
Dla **k2** (dzielenie próbne) `schedule(guided)` (2,394 s) i `schedule(dynamic)` (2,475 s) są zbliżone i znacznie lepsze od `schedule(static)` (6,339 s). Wynika to z nierównomiernego kosztu sprawdzania poszczególnych liczb — statyczny podział przydziela jednym wątkom wyłącznie duże liczby (droższe), innym — małe (tańsze), powodując silne niezrównoważenie.

Dla **k4 i k4a** (sito funkcyjne) `schedule(dynamic)` okazuje się lepszy od `schedule(guided)`, gdyż zadania (kolejne liczby pierwsze do wykreślenia) mają bardzo zróżnicowany koszt — mała liczba pierwsza (np. 2) generuje wiele wielokrotności, duża (np. 9999991) — nieliczne. Dynamiczny przydział po jednym zadaniu na raz najlepiej to wyrównuje.

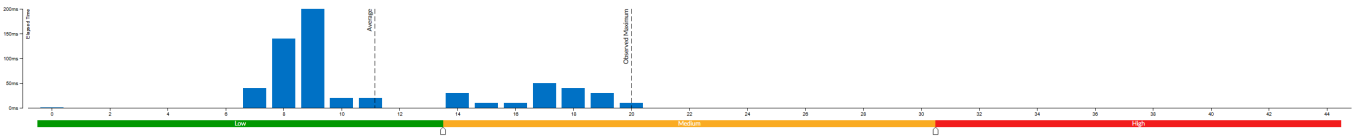
Dla **k5** (sito domenowe) `schedule(guided)` (0,060 s) jest najlepsze, bo bloki danych mają zbliżony koszt przetwarzania, a malejące paczki zadań redukują narzut synchronizacji przy końcu pętli, kiedy wątki kończą pracę w różnym czasie.

Porównanie wyników przetwarzania kodów k3 i k3a.

W celu głębszej analizy przyczyn różnicy w czasie przetwarzania między k3 (0,660 s) a k3a (0,130 s) przeprowadzono dodatkowy eksperyment z wykorzystaniem testu **Memory Access**. Wyniki przedstawiono na Rysunkach 1 i 2 oraz w Tablicy 6.



Rysunek 1: Wynik testu Memory Access BandwitdhHistogram dla kodu k3.



Rysunek 2: Wynik testu Memory Access BandwitdhHistogram dla kodu k3a.

Możemy zobaczyć, że średnia przepustowość k3 jest praktycznie 3 razy większa niż k3a. Są to około 32 GB/s, dochodząc nawet do 38GB/s. W kodzie k3 średnia wartość to już 11GB/s, ilość danych przechodzących przez magistralę jest znacznie mniejsza. Wynika to z naszej zmiany w kodzie k3a - zmieniamy pojedynczą iterację na przetwarzającą blok, a nie konkretną liczbę, dzięki czemu aktywny rozmiar tablicy mieści się w L1. W k3 zaś wykreślamy dla każdej iteracji liczby na całym zakresie, przez co każdy zapis trafia w odległe od siebie miejsca w pamięci.

	k3	**k3a**
P-Core L1 Bound	0.0%	0.0%
P-Core L2 Bound	1.1%	0.0%
P-Core L3 Bound	1.5%	0.0%
P-Core Store Bound	82.6%	0.0%
E-Core L1 Bound	0.0%	1.6%
E-Core L2 Bound	0.0%	0.0%
E-Core L3 Bound	0.0%	0.0%

Tablica 6: Porównanie czasów oczekiwania Memory Bound dla kodów k3 i k3a

Aby lepiej zrozumieć problem przyjrzeliśmy się również przyczynom ograniczeń wydajności dla obu kodów. Możemy zobaczyć, że k3 jest w 82,6% ograniczony przez oczekiwanie na przepełniony store buffer. Jest on więc dla naszego kodu wąskim gardłem — liczba żądań zapisu przerasta pojemność bufora zapisu procesora. Możemy też zauważyć że w kodzie k3 ograniczenia pamięci występują na L2 i L3; wynika to z tego że pętla wykreślająca wielokrotności przechodzi przez całą tablicę `result` o rozmiarze 100 mln elementów. Tablica ta nie mieści się w L1 ani L2 pojedynczego rdzenia, więc przy każdym dostępie procesor musi załadować odpowiednią linię cache z L2 lub L3.

W kodzie k3a obraz jest zupełnie odmienny. Przetwarzanie blokowe sprawia że aktywny obszar tablicy wynosi zawsze 32 kB i powinien mieścić się w L1 zarówno E-core jak i L1 P-core. Dla każdego bloku wszystkie dzielniki $i \leq \sqrt{\text{high}}$ są przetwarzane zanim algorytm przejdzie dalej - linie cache pozostają w L1 przez cały czas przetwarzania bloku i są wielokrotnie wykorzystywane. Stąd zerowe L2 Bound, L3 Bound i Store Bound - jedyne ograniczenie występuje na E-Core L1 Bound, następuje lekkie przepełnienie na pierwszej pamięci cache, czego jednak można było się spodziewać, jest ona mniejsza niż L1 P-core, mając dokładnie 32kB zamiast 48kB jak w wypadku P-Core.

Analiza wyników przetwarzania kodów k4 i k4a.

W tym eksperymencie poddaliśmy teście **Memory Access** kody k4 i k4a, aby zobaczyć jak dodanie warunku `if result[j-m]` wpływa na ograniczenia związane z pamięcią, i tym samym czas wykonania kodu. Wyniki przedstawiono w Tablicy 7.

	k4	**k4a**
P-Core Memory Bound	81.4%	28.3%
P-Core L1 Bound	2.9%	7.3%
P-Core L2 Bound	1.3%	0.8%
P-Core L3 Bound	3.6%	4.6%
P-Core Store Bound	72.3%	0.3%
E-Core Memory Bound	77.8%	58.1%
E-Core L1 Bound	0.6%	1.5%
E-Core L2 Bound	0.0%	0.2%
E-Core L3 Bound	0.0%	0.3%
Loads	94,902,847	389,711,691
Stores	241,807,254	155,704,671

Tablica 7: Porównanie czasów oczekiwania Memory Bound dla kodów k4 i k4a

Kod **k4** na obu rodzajach rdzeni uzyskał wyższe Memory Bound niż **k4a**. Na rdzeniu Performance możemy ponownie jak w wypadku kodu **k3** zobaczyć wysoki Store Bound (72.3%), który wiąże się z dużą liczbą operacji zapisu do tablicy `result` oraz ponownie przemieszczaniu się po całej tablicy. Stosunek Loads do Stores dla k4 wynosi około 1:2, odbywa się więcej zapisów niż odczytów, wielokrotnie bowiem nadpisujemy wartość `false` pod ten sam indeks. Dodanie warunku `if (result[j-m])` w kodzie k4a redukuje liczbę zapisów, zmieniając proporcje Loads/Stores do około 2.5:1, znacząco redukuje to Store Bound i poprawia wyniki na Memory Bound. Nie usuwa jednak problemu w całości, wciąż występują ograniczenia na pamięciach cache L1, L2, i L3. Dodanie warunku tworzy nowy koszt w postaci dodatkowych odczytów, sprawiając, że choć udało nam się zredukować Store Bound, kod wcale nie jest wolny od ograniczeń związanych z pamięcią i wykazuje się jedynie niewielką poprawą czasu wykonania.

Wyniki eksperymentu wyznaczania wielkości bloku dla kodów k3a i k5.

Aby wyznaczyć optymalną wielkość bloku dla kodów k3a i k5 napisaliśmy eksperyment, który wywoływał te kody dla standardowego zakres <2; max> z różnymi rozmiarami bloków od 16B do ilości liczb w zakresie z krokiem x2. Poniżej znajduje się część tabeli wynikowej, w której na środku znajduje się nasze poszukiwane optimum.

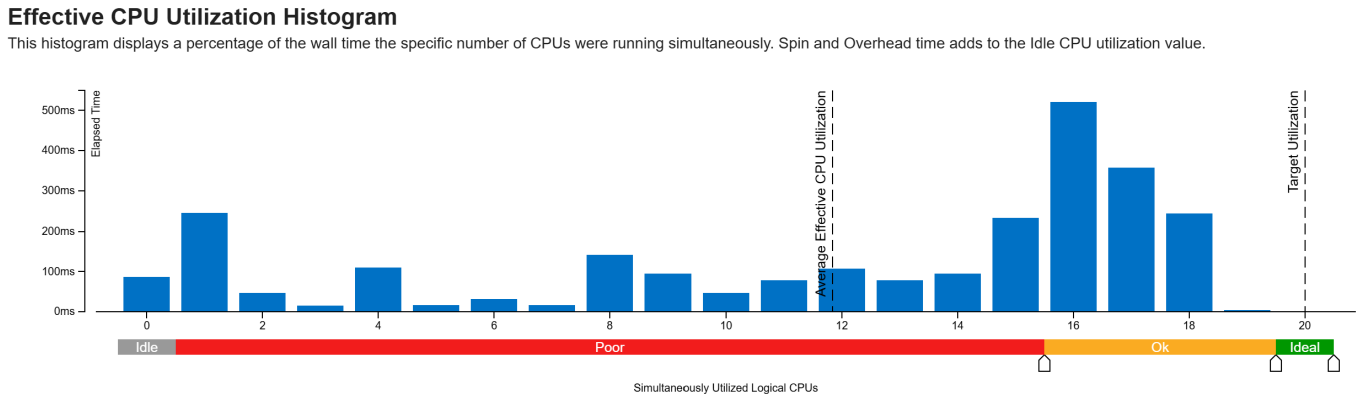
Blocksize	**k3a**	**k5**
4096	0.382 s	0.061 s
8192	0.229 s	0.040 s
16384	0.145 s	0.036 s
32768	0.107 s	0.030 s
65536	0.120 s	0.031 s
131072	0.121 s	0.039 s
262144	0.126 s	0.035 s
524288	0.128 s	0.040 s

Tablica 8: Wpływ rozmiaru bloku na czas przetwarzania kodów k3a i k5.

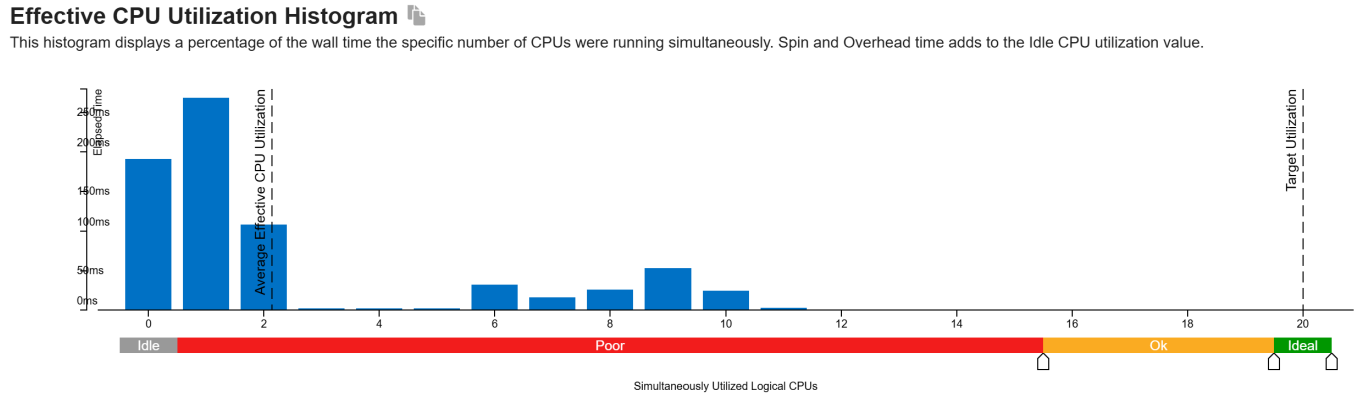
Optymalny rozmiar bloku wynosi **32768 elementów** (32 kB) dla obu kodów k3a i k5. Jest to wynik, którego możemy się spodziewać po sprawdzeniu parametrów procesora: choć posiada on ogółem 1 MB na pamięci L1, podzielona jest ona między 12 rdzeni, z czego 8 to rdzenie Performance z 48 kB cache L1, a pozostałe 4 to rdzenie Efficient z 32 kB cache L1. Z tych względów rozmiar bloku 32 kB będzie optymalny, będzie mieścił się w cache L1 obu typów rdzeni, jednocześnie będąc na tyle dużym aby zredukować niepotrzebny narzut związany z pobieraniem mniejszych ale większej liczby bloków z pamięci głównej do cache. Większy blok nie mieściłby się już zaś w L1, zmuszając do przenoszenia danych do wolniejszej pamięci L2, co ponownie zaczyna skutkować spadkiem wydajności.

Efektywne wykorzystanie CPU dla kodów równoległych.

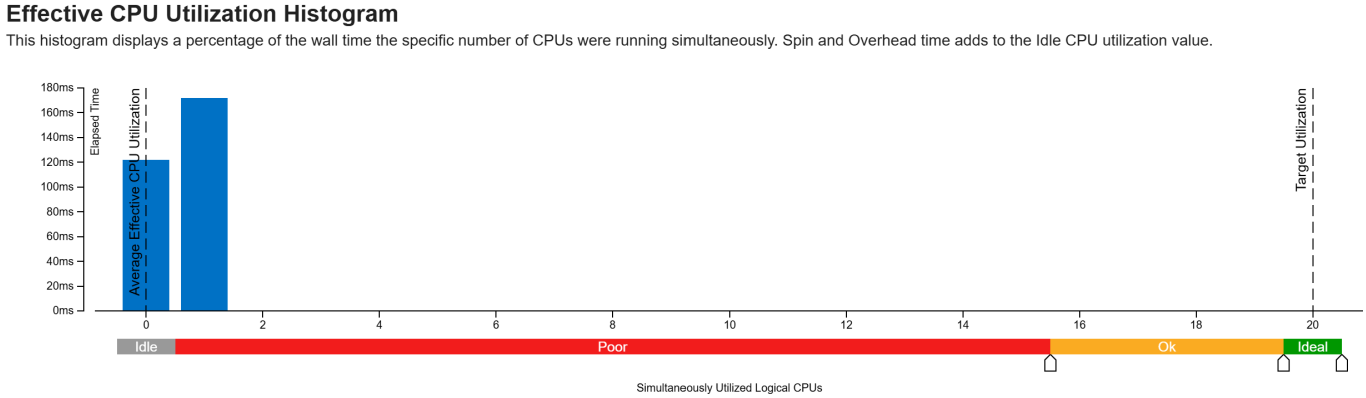
W ramach tego eksperymentu przeprowadzono test **Hotspots** dla kodów k2, k4a i k5. Wyniki przedstawiono na Rysunkach 3,4 i 5.



Rysunek 3: Wynik testu hotspots k2 dla zakresu <2; 100 mln>.



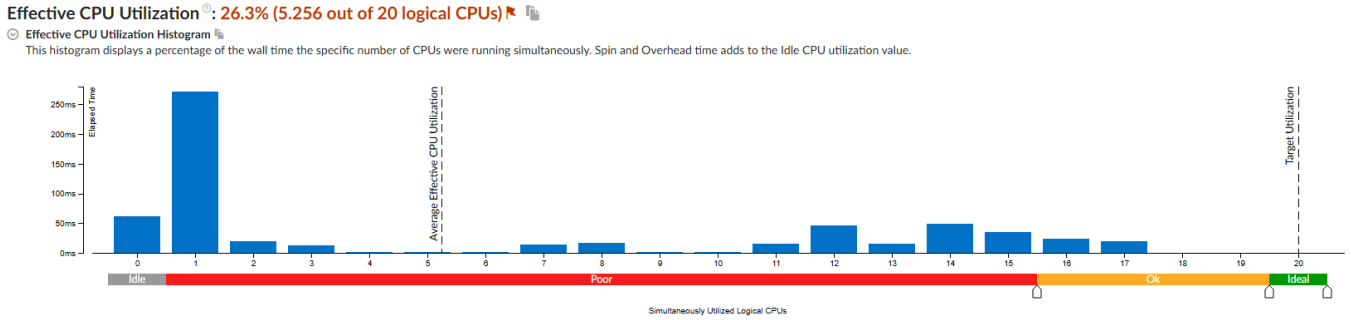
Rysunek 4: Wynik testu hotspots k4a dla zakresu <2; 100 mln>.



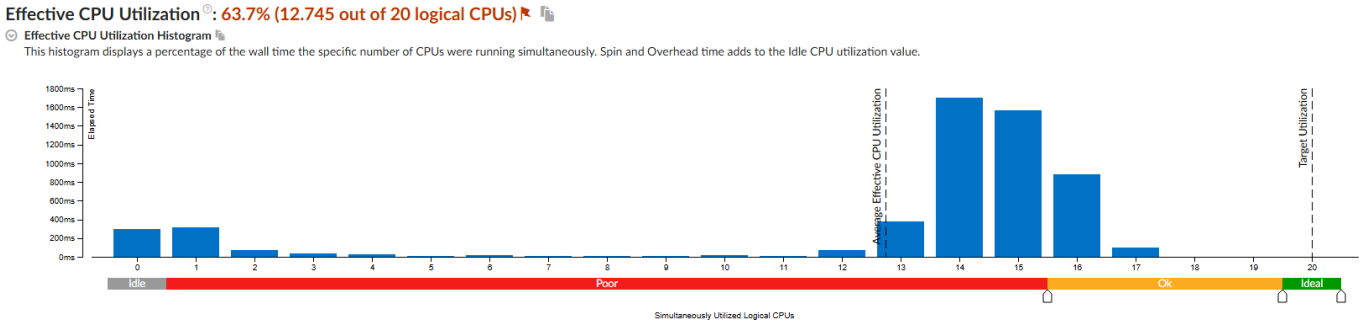
Rysunek 5: Wynik testu hotspots k5 dla zakresu <2; 100 mln>.

Dodatkowy eksperyment: zwiększenia zakresu liczb do 1 miliarda.

W czasie przetwarzania kodów równoległych w standardowym zakresie <2;100 mln> zauważyliśmy, że czasy przetwarzania są bardzo krótkie, co utrudnia dokładną analizę wydajności. Wykresy z sekcji wcześniej też nie wyglądały na zgodne z naszymi oczekiwaniami. Dlatego zdecydowaliśmy się na dodatkowy eksperyment, w którym zwiększyliśmy zakres liczb do 1 miliarda. Poniżej przedstawiamy porównania wyników testu **Threading** dla kodu równoległego k4.



Rysunek 6: Wynik testu threading k4 dla zakresu <2; 100 mln>.



Rysunek 7: Wynik testu threading k4 dla zakresu <2; 1 mld>.

Pierwsze co rzuca się przy analizie wyżej przedstawionych wykresów to znaczny wzrost efektywności CPU z 26.3% na 63.7% po zwiększeniu zakresu. Czas spędzony na pracy 0 lub 1 wątku pozostaje w obu wypadkach taki sam: w sumie około 0.6 s. Cały dodatkowy czas wykonania to czas spędzony w sekcji równoległej, który wzrósł z 0.267 s na 4.871 s. Z wykresu widać że najwięcej czasu kod wykonuje jednocześnie 14 wątków, a nie 1 jak w przypadku rysunku 1. Podobne wyniki zaobserwowaliśmy dla reszty równoległych kodów. Widzimy więc, że niska wydajność kodów nie wynika z ich złej implementacji, a zbyt małego zakresu liczb w teście, który nie pozwalał na efektywne wykorzystanie potencjału równoległego procesora.

4 Wnioski.

Na podstawie zebranych danych analitycznych z oprogramowania Intel VTune Profiler oraz pomiarów czasu wykonania (Elapsed Time), przeprowadzono wielokryterialną ocenę zaproponowanych implementacji.

4.1 Porównanie jakości i prędkości przetwarzania w badanych podejściach

Jakość rozwiązań, mierzona jako stosunek wielkości prędkości przetwarzania (wyrażonej w milionach sprawdzonych liczb na sekundę), ukazuje gigantyczną przepaść między rozwiązaniami naiwnymi a zoptymalizowanymi.

- **Ewolucja algorytmiczna (k1 vs k3):** Zmiana samego podejścia matematycznego z dzielenia próbnego (**k1**) na klasyczne Sito Eratostenesa (**k3**), nawet bez optymalizacji pamięciowej, doprowadziła do skrócenia czasu przetwarzania z 23,750 s do 0,660 s. Oznacza to, że metoda oparta na wykreślaniu wielokrotności jest **36 razy szybsza** od metody opartej na dzieleniu.
- **Wpływ zrównoleglenia w dzieleniu próbnym (k1 vs k2):** Implementacja równoległa (**k2**) pozwoliła na blisko 10-krotne skrócenie czasu (do 2,394 s). Mimo to, podejście oparte na dzieleniu nawet w wersji wielowątkowej pozostaje drastycznie wolniejsze od sekwencyjnego sita (**k3**), co podkreśla prymat doboru właściwego algorytmu nad czystą siłą obliczeniową.
- **Wpływ zrównoleglenia funkcyjnego w sicie (k3 vs k4/k4a):** Warianty równoległe bez lokalności danych (**k4**, **k4a**) osiągnęły lepsze czasy niż bazowe sito sekwencyjne (**k3**), redukując czas do odpowiednio 0,381 s i 0,296 s. Pokazuje to skuteczność paralelizacji OpenMP nawet przy obciążeniu magistrali pamięciowej.
- **Wpływ lokalności danych (k3 vs k3a):** Porównanie wariantów sekwencyjnych sita ukazuje fundamentalne znaczenie dostępu do pamięci. Segmentacja tablicy na bloki w kodzie **k3a** (Listing 4) pozwoliła na osiągnięcie prędkości 35,714 mln l./s wobec 14,394 mln l./s w **k3**. Przetwarzanie w wariancie **k3a** trwało **5,07 raza krócej** wyłącznie dzięki dopasowaniu rozmiaru przetwarzanych danych do rozmiaru pamięci podręcznej, bez jakiejkolwiek zmiany złożoności obliczeniowej.
- **Wpływ zrównoleglenia domenowego (k3a vs k5):** Wprowadzenie wielowątkowości do zoptymalizowanego algorytmu sekwencyjnego (**k3a** $\rightarrow$ **k5**) zwiększyło prędkość przetwarzania o około **168%** (z 35,714 do 96,024 mln l./s), redukując czas z 0,130 s do zaledwie 0,060 s (**2,16 raza krócej**).

4.2 Efektywność wykorzystania struktur wewnętrznych procesora i analiza wąskich gardeł

Użyty w badaniach procesor Intel Core i7-12700F posiada architekturę hybrydową, oferując 8 rdzeni typu Performance (P-Core, 48 kB L1 Data Cache) oraz 4 rdzenie typu Efficient (E-Core, 32 kB L1 Data Cache). Analiza profilowania VTune (Tabele 4, 6 i 7) ukazuje, jak poszczególne kody radzą sobie z tą strukturą:

- **Krytyczne wąskie gardło pamięci (Memory Bound) w kodzie k3:** Jak wynika z Tablicy 3, kod **k3** na rdzeniach P-Core wykazuje 100% ograniczeń typu *Back-End Bound*, z czego równe 100% to *Memory Bound*. Tablica 6 precyzuje, że aż **82,6% cykli to oczekiwanie na bufor zapisu (Store Bound)**. Jest to bezpośrednia konsekwencja linii 15 w kodzie **k3** (`result[j-m] = false;`), gdzie skok do pamięci o wartości i w tablicy o rozmiarze 100 mln elementów powoduje ciągłe chybienia (cache misses) w pamięciach L1, L2 i L3, zmuszając procesor do bezpośredniego zapisu do wolnej pamięci RAM.
- **Rozwiązanie problemu w kodzie k5:** Wariant domenowy z lokalnością danych (Listing 7) idealnie utylizuje sprzęt. Zastosowanie bloku o rozmiarze 32768 elementów (32 kB) sprawia, że cały aktywny segment tablicy `result` mieści się w najmniejszej pamięci podręcznej układu – L1 Data Cache rdzeni E-Core (32 kB). Dzięki temu, parametry *Memory Bound* dla **k5** spadły do zaledwie **16,7%** (P-Cores) i **18,8%** (E-Cores). Procesor zamiast czekać na dane, mógł skupić się na obliczeniach (*Retiring* na poziomie 25%). Zwiększenie rozmiaru bloku (eksperyment w Tabeli 8) ponad ten próg np. do 131072 (128 kB) powoduje spadek wydajności i wydłużenie czasu pracy o blisko **30%** (z 0,030 s do 0,039 s), gdyż blok wypada z najszybszej pamięci L1.
- **Próba usunięcia redundancji w kodach k4 i k4a:** W kodzie **k4** wiele wątków nieustannie nadpisuje te same komórki pamięci wartością `false` (tzw. *false sharing*). Dodanie warunku `if (result[j-m])` w kodzie **k4a** miało na celu redukcję zapisów. Tablica 7 udowadnia, że operacja ta zmniejszyła *P-Core Store Bound* z **72,3% do zaledwie 0,3%**, redukując liczbę zapisów (Stores) o **35,6%**. Niestety, operacja warunkowa wymagała drastycznego wzrostu liczby odczytów (Loads) z 94,9 mln do 389,7 mln (wzrost o **310%**). W efekcie kod **k4a** przetwarza dane **1,28 raza szybciej** niż **k4**, ale wciąż jest dławiony przez limity przepustowości pamięci na wszystkich poziomach (L1, L2, L3).

4.3 Ocena jakości zrównoleglenia i skalowalności

W celu oceny zrównoleglenia przeanalizowano parametry przyspieszenia oraz efektywności dla rdzeni fizycznych (Tablica 2) oraz wyniki testów Hotspots i Threading.

- **Przyspieszenie i efektywność:** Dla optymalnego wariantu **k5**, przyspieszenie względem najlepszego kodu sekwencyjnego (**k3a**) wynosi **2,17**. Biorąc pod uwagę użycie 12 rdzeni fizycznych, efektywność wynosi **0,18** (18%). Jest to wartość relatywnie niska, sugerująca, że przeważająca część zasobów pozostaje niewykorzystana (współczynnik *Effective CPU* wynosi zaledwie 23,4%).

- **Skalowalność i wykorzystanie procesorów logicznych:** Analiza wyników Hotspots (Rysunki 3, 4, 5) oraz Threading (Rysunek 6) pokazuje, że system faktycznie angażuje wiele rdzeni. Warto zauważyć, że przy większych obciążeniach wykorzystujemy faktycznie te 15-17 wątków na raz (co widać na wykresie threading dla **k4**), zatem zmniejszenie ich liczby miałoby realny, negatywny wpływ na czas przetwarzania kodów, mimo pozornie niskiej efektywności przy małych zakresach.

- **Przyczyny niskiej efektywności i eksperyment ze skalowalnością:** Niska utylizacja nie jest wynikiem błędów w architekturze kodu **k5**, lecz narzutu środowiska uruchomieniowego (tworzenie i synchronizacja wątków OpenMP) na zbyt małej instancji problemu. Czas wykonania 0,060 s jest tak krótki, że narzut administracyjny przewyższa czas obliczeń użytecznych. Eksperyment z zakresem powiększonym 10-krotnie (do 1 mld, Rysunki 3 i 4) dobitnie to udowadnia: współczynnik *Effective CPU* wzrasta tam drastycznie z 26,3% do **63,7%**. Oznacza to, że efektywność zrównoleglenia wariantu domenowego skaluje się silnie i pozytywnie wraz ze wzrostem wolumenu danych poddawanych analizie.

4.4 Określenie najlepszego i najgorszego podejścia

1. **Najlepsze podejście: k5 (Równoległe sito domenowe z lokalnością danych).** To rozwiązanie charakteryzuje się największą prędkością przetwarzania (96,024 mln l./s), deklasując najgorszą metodę o blisko **2280%** (prędkość 22,8 raza większa od metody **k1**). Przewaga ta wynika z dwóch filarów: matematycznego (niska złożoność $O(n \log \log n)$) oraz sprzętowego (praca na blokach 32 kB zapobiega obciążeniu magistrali pamięciowej, a każda zmiana w L1 Cache pozostaje na lokalnym rdzeniu, unikając inwalidacji protokołów spójności pamięci typowej dla *false sharing*).

2. **Najgorsze podejście: k1 (Sekwencyjne dzielenie próbne).** Metoda ta uzyskała prędkość rzędu 4,210 mln l./s (czas 23,750 s). Jej zła wydajność jest niezależna od wykorzystania pamięci (nie występuje tu problem cache misses). Prawdziwą przyczyną jest kolosalna liczba wykonanych instrukcji procesora. Jak pokazuje Tablica 2, procesor musiał wycofać (Instructions Retired) aż **405,7 miliardów instrukcji** – co jest wartością **522 razy większą** niż w kodzie **k3a** (0,777 mld). Operacja modulo występująca w podwójnej pętli (Listing 1) jest najdroższą sprzętowo operacją arytmetyczną w jednostce ALU.

4.5 Analiza wpływu polityk szeregowania (Scheduling) na wydajność

Wpływ wyboru dyrektywy OpenMP różni się radykalnie w zależności od natury realizowanego problemu (Tablica 5):

- **Problem niezrównoważony kosztowo (k2):** W kodzie **k2** iteracje są skrajnie nierówne (liczba 99 999 999 wymaga wielokrotnie więcej dzieleń próbnych niż liczba 50). Szeregowanie `static` z góry przydziela iteracje, przez co wątek z najdroższymi obliczeniami blokuje resztę. Zmiana na `guided` skraca czas z 6,339 s do 2,394 s (**2,6 raza szybsze wykonanie**), ponieważ wielkość przypisywanych porcji roboczych maleje proporcjonalnie do zbliżania się ku końcowi pętli, perfekcyjnie balansując czas ukończenia wszystkich wątków (Effective CPU 68,9%).

- **Zjawisko fałszywego współdzielenia (k4 i k4a):** W wariancie **k4** to `dynamic` z rozmiarem paczki (chunk) 1 wypada najlepiej (0,296 s wobec 0,551 s dla `static`). Jest tak, ponieważ poszczególne liczby pierwsze wykreślają drastycznie różną liczbę wielokrotności (np. dla $i = 2$ iteracji pętli wykreślającej jest miliony, a dla bardzo dużych i zaledwie kilka). `Dynamic, 1` gwarantuje ciągłe "karmienie" rdzeni zadaniami, bez względu na koszt wykreślania dla konkretnego i.

- **Problem wysoce zrównoważony (k5):** Blokowe wykreślanie zapewnia bardzo powtarzalny czas pracy w każdej iteracji, dlatego `guided` osiąga tutaj najlepszy czas (0,060 s), redukując koszty zarządzania kolejką wątków w porównaniu do klasycznego przydziału `dynamic`.

Podsumowanie ogólne: Wyciśnięcie maksymalnej wydajności z hybrydowego procesora wielordzeniowego nie polega na ślepym dodaniu dyrektywy `#pragma omp parallel for`. Najlepszy k od **k5** dowodzi, że optymalizacja musi zachodzić kaskadowo: najpierw redukcja złożoności matematycznej (Sito), następnie optymalizacja lokalności danych w pamięci układu (blok 32 kB dostosowany do rozmiaru L1 układów E-Core, eliminacja *Store Bound*), a dopiero na końcu zastosowanie podziału wielowątkowego (domenowego), który chroni przed wyścigami danych w linii cache (*False Sharing*). Dopiero taka synergia pozwoliła osiągnąć skok prędkości o **ponad 2000%**.